INFORMATION RETRIEVAL & NATURAL LANGUAGE PROCESSING CASE STUDY

How Zomato Improved Search Relevance Using NLP Intent Identification

Architectural decomposition based on the engineering dissection by Arpit Bhayani (Video ID: JL9x9N6YSUc) & Zomato Engineering.

THE SEARCH DILEMMA IN FOOD AGGREGATION

Food discovery platforms operate on open search inputs where users submit non-homogeneous, multi-entity queries such as `"best Domino's Pizza near me"` or `"Jack's Aloo Tikki Burger"`. Firing raw multi-word strings directly into search engines (e.g., Elasticsearch or Solr) causes catastrophic relevance failures: full-text matching prioritizes title token matches, promoting restaurants whose brand name happens to include stop-words or dish names over actual culinary targets. By introducing an **Intent Identification Gateway** powered by **BPE**, **Word2Vec**, and **BiLSTM + CRF**, Zomato deconstructs raw queries into discrete entities (Restaurant, Dish, Location), routing structured queries to dedicated indexes.

1. THE FOOD DISCOVERY LANDSCAPE: AGGREGATOR ENTITIES

As a hyper-scale food delivery and restaurant aggregator, Zomato's primary conversion funnel hinges entirely on its search bar. Users interact with the platform to discover and order items across three fundamental domain entities:

- **Dish / Food Item:** Specific menu items or culinary preparations (e.g., *Pizza*, *Burger*, *Paneer Butter Masala*, *Dal Makhni*, *Chai*, *Samosa*).
- **Restaurant Brand / Chain:** Merchant identifiers and restaurant listings (e.g., *Domino's*, *Pizza Hut*, *McDonald's / MCD*, *Jack's*).
- **Cuisine / Category:** Broader culinary classifications (e.g., *Chinese*, *North Indian*, *Vegetarian / Veg*).

Because the search box is an unconstrained free-form input, users routinely concatenate these discrete entities into a single query string without formal delimiters (e.g., `"Pizza Dominos"`, `"MCD Burger"`).

2. NAIVE SEARCH ENGINE FAILURE MODES (ELASTICSEARCH / SOLR)

In standard search engine deployments (Elasticsearch or Apache Solr), document retrieval relies on inverted indexes and field-level term frequency-inverse document frequency (TF-IDF) or BM25 scoring with static field weights. Typical document schemas separate data into distinct indexes:

- **`restaurants` Index:** Documents contain `title` (restaurant name), `description`, `cuisine_list`, and `address`. Standard boost configurations assign heavy weight to titles (e.g., $w_{ ext{title}} = 10$) and lower weight to general descriptions ($w_{ ext{description}} = 1$).
- **`food_menu` Index:** Documents represent individual dishes with fields such as `dish_name`, `dish_description`, `ingredients`, and parent `restaurant_id`.

Naive Direct Execution: "best coffee near me"

When the query `"best coffee near me"` is fired directly against the `restaurants` index with $w_{ ext{title}} = 10$:

- Elasticsearch tokenizes into `["best", "coffee", "near", "me"]`.
- Any merchant with "Best", "Coffee", or "Near" in its title ranks highest:
  1. *Best Coffee Cafe* (matches 2 title tokens)
  2. *Best Bliss* (matches "best")
  3. *Bar Best* (matches "best")
- **Relevance Disaster:** The user wanted top-rated coffee nearby, but the search engine returned random venues whose brand names match common adjectives.

Intent-Aware Execution: Entity Decomposition

When the query is routed through an Intent Identification model first:

- Extracts: **Dish: Coffee**, **Intent: Near Me (Geo)**, **Modifier: Best (Rating > 4.0)**.
- Constructs targeted multi-index Elasticsearch query:
  - Filters `food_menu` index where `dish_name = "coffee"`.
  - Applies Elasticsearch `geo_distance` filter using user coordinates.
  - Sorts merchants by aggregate rating and proximity.
- **Outcome:** Serves authentic, nearby cafes brewing quality coffee.

3. THE VOICE SEARCH INFLECTION: VERBOSE & CONVERSATIONAL INPUTS

The relevance challenge expanded dramatically with the introduction of **Voice Search**. While typing encourages terse, truncated queries due to user effort, voice speech-to-text (STT) produces verbose, natural-language conversational statements:

Dual-Dish Voice Queries

`"garlic bread with cheese dip"`

Contains two distinct dish entities (*garlic bread* and *cheese dip*). Firing blindly confuses whether the user wants a single menu item or a restaurant serving both.

Category + Location Voice Queries

`"veg restaurant koramangala"`

Combines a dietary preference/category (*veg*) with a geographic neighborhood (*koramangala*). Token match ranks venues named "Koramangala Veg" first.

Multi-Dish Combo Voice Queries

`"chai and samosa"`

Requires joint co-occurrence across the restaurant menu rather than returning tea stalls without snacks or sweet shops without tea.

4. QUERY CLASSIFICATION TAXONOMY

Empirical query log analysis reveals three dominant composite patterns in food search:

5. CORE PRODUCTION CHALLENGES IN FOOD SEARCH NLP

Applying standard supervised natural language processing to food delivery queries encounters four severe real-world constraints:

Query Pattern	Canonical Query Example	Extracted Entities & Structural Meaning	Execution Strategy
Dish + Dish	`"pizza with cheese dip"`	`[Dish: Pizza]` + `[Dish: Cheese Dip]`	Query `food_menu` index for co-occurring menu items in single merchant.

1. Total Lack of Labeled Ground-Truth Data

Millions of arbitrary queries arrive daily without human-annotated entity boundaries. Hand-labeling is prohibitively expensive and cannot keep pace with new restaurant names and trendy culinary items. Models must learn semantic associations via unsupervised corpus pre-training.

2. Multilingual & Code-Mixed Queries (Hinglish)

Users speak across 30+ regional languages in India, frequently mixing Hindi grammatical connectives with English culinary terms (e.g., `"dal makhni ke saath naan"`, `"ek pizza aur do coke"`). Filler words (*ke saath*, *aur*) must be ignored while isolating anchor entities.

3. Semantic Equivalence & Synonym Resolution

Colloquial phrases express identical intent: `"MCD Burger"` and `"MCD ke Burger"` mean the exact same thing. Culinary terms exhibit local synonyms: `"rajma rice"` $\equiv$ `"rajma chawal"`. The engine must map regional culinary nomenclature into common semantic spaces.

4. Phonetic & Tokenization Variations

Transliteration creates multiple valid spellings: *rumali roti* vs. *roomali roti*. Furthermore, speech-to-text engines concatenate tokens without whitespace (e.g., `"friedrice"` vs. `"fried rice"`), defeating naive whitespace-splitting tokenizers.

6. THE MACHINE LEARNING STACK: FROM CHARACTERS TO STRUCTURED ENTITIES

To overcome these challenges without relying on fragile hard-coded regexes, Zomato deployed a unified four-stage machine learning architecture combining subword tokenization, vector embeddings, and sequential deep learning:

The Zomato NLP Intent Identification Inference Pipeline

`[Raw Query: "Jack's Aloo Tikki Burger"]`

↓

1. Byte Pair Encoding (BPE): Subword Tokenization → `["Jack's", "Aloo", "Tikki", "Burger"]`

↓

2. Domain Word2Vec: Dense Vector Embeddings → $ec{v} \in \mathbb{R}^{128 ext{ to } 256}$ (captures culinary semantics)

↓

3. Bi-directional LSTM: Sequence Context Encoding (Forward $ec{h}_t$ & Backward $\overleftarrow{h}_t$ hidden states)

↓

4. Conditional Random Fields (CRF): Global Tag Sequence Optimization → `[B-REST, B-DISH, I-DISH, I-DISH]`

Stage 1: Byte Pair Encoding (BPE) for Robust Subword Tokenization

Naive tokenization splits strings at whitespace characters (e.g., `text.split(" ")`). In production, this fails when voice STT produces concatenated words (`"friedrice"`) or out-of-vocabulary (OOV) regional dishes. **Byte Pair Encoding (BPE)** is an unsupervised, frequency-based compression algorithm that iteratively merges the most frequent pairs of adjacent characters or subwords in the corpus.

- **OOV Immunity:** If an unseen compound word appears, BPE decomposes it into known subword constituent tokens (e.g., `"paneer"` + `"tikka"`), preserving semantic interpretability.
- **Spelling Variation Resilience:** Handles phonetic variations (*rumali* / *roomali*) by preserving shared root subwords.

Stage 2: Word2Vec Dense Vector Embeddings

Computers operate on numbers, not arbitrary text strings. Assigning static scalar integers (e.g., `Burger = 1`, `Pizza = 2`) provides zero mathematical representation of semantic relationships. Zomato trains a **Word2Vec** neural network (Continuous Bag-of-Words or Skip-Gram) directly on its proprietary domain corpus (restaurant names, complete food menus, user reviews, and geographic locations).

THE VECTOR SPACE GEOMETRY OF CULINARY ENTITIES

Word2Vec maps every vocabulary token into an N-dimensional continuous vector space ($d = 128 ext{ or } 256$):

`Classic NLP Geometry: $ec{v}( ext{King}) - ec{v}( ext{Man}) + ec{v}( ext{Woman}) pprox ec{v}( ext{Queen})$`

`Zomato Food Geometry: $ec{v}( ext{rajma chawal}) pprox ec{v}( ext{rajma rice}) \quad ext{and}\quad ext{cosine}(ec{v}( ext{roti}), ec{v}( ext{naan})) pprox 1.0$`

Because semantically similar food items appear in identical contexts across menus, their vector dot products are maximized, allowing the model to naturally identify synonyms and culinary equivalents.

Stage 3: Bi-directional Long Short-Term Memory (BiLSTM)

Word vectors represent static semantics, but word meaning depends heavily on sequential context. In the query `"Jack's Aloo Tikki Burger"`, whether *"Jack's"* is a restaurant brand or part of a dish depends on the tokens that follow it. A standard forward LSTM only reads tokens left-to-right ($ec{h}_t$), missing future context.

A **Bi-directional LSTM (BiLSTM)** processes the sequence in two simultaneous directions:

- **Forward Pass ($ec{h}_t$):** Captures preceding context (e.g., observing that "Jack's" is followed by culinary descriptors).
- **Backward Pass ($\overleftarrow{h}_t$):** Captures succeeding context (e.g., observing that "Burger" is preceded by "Aloo Tikki" and a brand prefix).

- **Concatenation:** The final token representation $h_t = [\overrightarrow{h}_t ; \overleftarrow{h}_t]$ encapsulates the complete bidirectional temporal sentence structure.

Stage 4: Conditional Random Fields (CRF) for Sequence Tagging

Feeding BiLSTM hidden states into a naive softmax classifier produces independent per-token predictions. This frequently

7. PRODUCTION SYSTEM ARCHITECTURE: THE "INTENT DETOUR"

Rather than executing a blunt pass-through from client search requests directly into Elasticsearch, Zomato introduced an intermediary **Intent Identification Detour** within its search infrastructure.

Legacy Direct Architecture (Flawed)

Client → Search Service → Direct Elasticsearch Query

- Raw string fired as full-text BM25 query across all indexes.
- Adjectives and location words pollute restaurant title weights.
- Zero semantic awareness of multi-entity composite intent.
- Frequent zero-result pages or absurdly irrelevant merchant listings.

Modern Intent Detour Architecture (Optimized)

Client → Search Service → NLP Gateway Detour → Targeted Query

- Search Service routes raw query string to dedicated NLP Gateway.
- Model parses entities: `{restaurant: "Jack's", dish: "Aloo Tikki Burger"}`.
- Search Service constructs specialized multi-index query targeting discrete fields.
- Elasticsearch returns pinpoint matches; UI highlights matching dishes.

End-to-End Search Infrastructure with Intent Identification Gateway

```
[Client Mobile App] → (1) User types / speaks "Jack's Aloo Tikki Burger"
  ↓
[Search Core Service] → (2) Detour API call: POST /intent/extract {query: "Jack's Aloo Tikki Burger"}
  ↓
[Inference API Gateway (EC2 / Flask)] → Runs BPE + Word2Vec + BiLSTM + CRF
  ↓
  Returns Structured Entities: { "restaurant": "Jack's", "dish": "Aloo Tikki Burger" }
  ↓
[Search Core Service] → (3) Synthesizes targeted Elasticsearch multi-index query:
  • Match restaurants.title: "Jack's"
  • Match food_menu.dish_name: "Aloo Tikki Burger"
  • Filter restaurants.geo_distance: user_location < 5km
  ↓
[Elasticsearch Cluster] → (4) Returns highly scored documents
  ↓
[Search Core Service] → (5) Formats response: Promotes "Jack's" with "Aloo Tikki Burger" badge highlighted → [Client App]
```

8. ARCHITECTURAL TRADE-OFFS & PERFORMANCE COMPARISON

System Dimension	Naive Full-Text Search (Direct ES)	NLP Intent Detour Architecture (Zomato)
Query Understanding	Lexical term matching (BM25/TF-IDF); blind to entity roles.	Semantic Named Entity Recognition (NER); tags Dish, Restaurant, Location.
Multi-Entity Handling	Pollutes ranking: brand name matches obscure dish items.	Separates entities; routes to dedicated `restaurants` and `food_menu` indexes.
Voice Search Compatibility	Extremely fragile; conversational filler words dilute score.	Robust; BPE subwords and CRF layers strip conversational stop-words.
Spelling & Regional Variations	Requires massive fuzzy edit distances; fails on synonyms.	Subword BPE handles OOV; Word2Vec geometry captures culinary synonyms.
Network & Latency Cost	Single round-trip to Elasticsearch (~15–25ms).	Added inference hop (+10–20ms); offset by caching frequent query intents.
Relevance & Conversion Rate	Poor on complex queries; causes user drop-off.	Drastic conversion increase; exact restaurant promoted with highlighted dish.

9. SYSTEMS ENGINEERING & ARCHITECTURAL TAKEAWAYS

CORE LESSONS FOR HIGH-SCALE SEARCH SYSTEMS

1. **Search Engines Are Index Stores, Not Language Models:** Distributed search engines like Elasticsearch and Solr excel at high-throughput inverted index traversal and BM25 document scoring. Expecting an index engine to infer human grammatical intent from unstructured multi-entity strings is an architectural antipattern.
2. **The Intent Detour Pattern:** Separating query understanding (NLP inference gateway) from document retrieval (search cluster) allows both layers to scale independently and evolve their underlying tech stacks without coupling.
3. **Domain-Specific Pre-Training Outweighs Generic Models:** Training Word2Vec directly on restaurant menus and food logs captures local culinary geometry (e.g., *rajma chawal* $ pprox$ *rajma rice*) that massive generic language models frequently fail to represent.
4. **Subword Encoding Eliminates Out-of-Vocabulary (OOV) Bottlenecks:** Employing Byte Pair Encoding (BPE) enables production systems to gracefully handle regional spelling variations, code-mixed dialects, and speech-to-text concatenation artifacts without manual dictionary curation.